

Ejercicio 14: Construcción Óptima de Puentes sin Cruces

Este problema es un clásico de Programación Dinámica muy interesante. Aunque a primera vista parece geométrico debido a las coordenadas en un mapa bidimensional con un río, el enunciado esconde una reducción matemática directa a uno de los problemas fundamentales de la algoritmia.

1. La intuición REAL (Reducción del Problema)

Analicemos qué significa geoméricamente que dos puentes no se crucen:

- Tenemos n ciudades en el sur ordenadas según su coordenada x . Para simplificar el análisis, reindexamos los puntos del sur de modo que sus posiciones físicas estén en orden creciente:

$$a(1) < a(2) < \dots < a(n)$$

- Las ciudades correspondientes en el norte tienen coordenadas x definidas por $b(1), b(2), \dots, b(n)$, pero no tienen por qué estar ordenadas.
- Solo se puede conectar la ciudad i del norte con la ciudad i del sur.
- Supongamos que decidimos construir los puentes para dos ciudades distintas, i y j , con $i < j$.
 - En el sur, al estar las ciudades ordenadas de forma creciente, la ciudad i está a la izquierda de la ciudad j , es decir, $a(i) < a(j)$.
 - Para que el puente que va de la ciudad i del sur a la i del norte no se cruce con el puente que va de la ciudad j del sur a la j del norte, obligatoriamente la ciudad i del norte debe estar a la izquierda de la ciudad j del norte.
 - Es decir, se debe cumplir estrictamente que:

$$b(i) < b(j)$$

Conclusión Matemática

Queremos elegir el mayor conjunto posible de índices $\{i_1, i_2, \dots, i_k\}$ que podamos conectar con puentes de modo que se cumpla al mismo tiempo:

- En el sur (ordenados por defecto): $i_1 < i_2 < \dots < i_k$
- En el norte: $b(i_1) < b(i_2) < \dots < b(i_k)$

Esto es exactamente la definición de la Subsecuencia Creciente de Máxima Longitud (SCML) (en inglés, *Longest Increasing Subsequence* o LIS) sobre la secuencia de las coordenadas del norte $B = \langle b(1), b(2), \dots, b(n) \rangle$.

2. Estado de la Programación Dinámica

Definimos el vector de estados $DP[i]$ como:

$DP[i]$ = La longitud del conjunto máximo de puentes no cruzados que se pueden construir considerando únicamente las ciudades de la sublista $1 \dots i$, obligando a construir el puente para la ciudad i .

Donde $1 \leq i \leq n$.

3. Casos Base

Para cualquier ciudad individual i , siempre podemos construir al menos un único puente independiente (el de ella misma):

$$DP[i] = 1 \quad \forall i \in \{1, 2, \dots, n\}$$

4. Ecuación de Recurrencia

Para calcular $DP[i]$ con $1 < i \leq n$, buscamos de entre todos los puentes anteriores j ($1 \leq j < i$) cuál de ellos puede conectarse antes de la ciudad i sin provocar cruces.

Para que el puente j no se cruce con el puente i , debe cumplirse que la coordenada del norte de j esté a la izquierda de la del norte de i :

$$DP[i] = 1 + \max \left\{ \{0\} \cup \{DP[j] \mid 1 \leq j < i \wedge b(j) < b(i)\} \right\}$$

5. Pseudocódigo Modo Examen

Este algoritmo calcula el número máximo de puentes y utiliza un vector `parent` para poder reconstruir hacia atrás qué puentes concretos deben construirse.

```

Funcion CalcularPuentesOptimos(b, n)
Inicio
    DP <- Vector de tamaño n + 1
    parent <- Vector de tamaño n + 1 // Guarda el puente previo óptimo

    // 1. Inicialización de Casos Base
    Para i <- 1 Hasta n Con Paso 1 Hacer
        DP[i] <- 1
        parent[i] <- -1
    FinPara

    // 2. Llenar el vector DP aplicando la recurrencia
    Para i <- 2 Hasta n Con Paso 1 Hacer
        Para j <- 1 Hasta i - 1 Con Paso 1 Hacer
            // Si la ciudad del norte j está a la izquierda de la i, no hay cruce
            Si b[j] < b[i] Entonces
                Si DP[j] + 1 > DP[i] Entonces

```

```

        DP[i] <- DP[j] + 1
        parent[i] <- j // El puente i se encadena detrás del puente j
    FinSi
FinSi
FinPara
FinPara

// 3. Buscar el máximo global en el vector DP
max_puentes <- 0
ultimo_puente_elegido <- -1

Para i <- 1 Hasta n Con Paso 1 Hacer
    Si DP[i] > max_puentes Entonces
        max_puentes <- DP[i]
        ultimo_puente_elegido <- i
    FinSi
FinPara

// 4. RECONSTRUCCIÓN: Recuperar qué puentes construir de izquierda a derecha
puentes_construir <- Lista vacía
nodo_actual <- ultimo_puente_elegido

Mientras nodo_actual != -1 Hacer
    Añadir nodo_actual al principio de puentes_construir
    nodo_actual <- parent[nodo_actual]
FinMientras

Imprimir("Número máximo de puentes sin cruzar: ", max_puentes)
Imprimir("Ciudades a conectar: ", puentes_construir)

Retornar max_puentes
FinFuncion

```

6. Análisis de Complejidad

- **Complejidad Temporal:** $\Theta(n^2)$. El algoritmo utiliza dos bucles anidados para rellenar el vector de estados DP . El externo se ejecuta n veces y el interno recorre para cada paso i los elementos j anteriores, realizando comparaciones elementales de coste constante $O(1)$. La posterior fase de reconstrucción del camino de puentes toma tiempo lineal $O(n)$.

$$\sum_{i=1}^n (i-1) = \frac{n(n-1)}{2} \implies \Theta(n^2)$$

(Nota teórica: Al reducirse a un LIS, es posible optimizar el tiempo a $O(n \log n)$ si en lugar del DP tradicional se utiliza una búsqueda binaria asistida con un vector de candidatos intermedios).

- **Complejidad Espacial:** $\Theta(n)$ para almacenar los vectores unidimensionales DP y $parent$ de tamaño $n + 1$.